CSCI 2810 | 10/20/26

shortest path algo on edit-like graph — linear time

score of max score path ≡ alignments

- longest path is intractable (brute force, exponential)

DAGs: shortest & longest path time in linear time. Key: topological order

→ Dijkstra's Algorithm (again?)

$G = \langle N, A \rangle$ directed graph

N = set of nodes/vertices = $\{1, 2, \ldots, n\}$, 1 is source node.

A = set of edges, each edge has non-negative cost

Problem: given $G = (N, A)$, $L[i,j]$ = cost of edge (i,j)

Compute the cost of the shortest paths from the source to each of the other nodes in G.

C, S two sets of nodes

"candidate" nodes

"special" nodes already chosen

At any step, S contains nodes whose minimal cost from source node is alr known. Rest are in C.

At any step we choose a candidate node in C where the cost to the source node is smallest cost and add it to S.

Def: a path from the source to a node v is special if all intermediate nodes $\in S$.

- Array D: contains the cost of the shortest special path to each node in G.
- When we add a new node v from C to S, then the shortest special path to v is also the shortest path in the graph

Let's look @ the Code

$N = \{1, 2, \ldots, n\}$

$L[i,j] \geq 0$ if $(i,j) \in A$

$L[i,j] = \infty$ if $(i,j) \notin A$

Initialization 1 = source node

$S = \{1\}$, $C = \{2, 3, \ldots, n\}$

For $i = 2$ to n do

$D[i] = L[1, i]$ what if not defined? ✓

$P[i] = 1$ $i = 3, \ldots, n$ (??)

Greedy loop

Repeat $n-2$ times

v = node in C w/ minimum value $D[v]$ ↳ "greedy"

$S = S \cup \{v\}$

$C = C \setminus \{v\}$

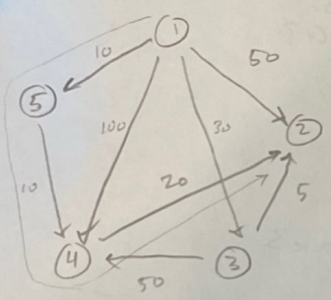
For each $w \in C$, do what if not defined? ↳ is ∞. Basically ignore

if $D[w] > D[v] + L[v,w]$

$D[w] = D[v] + L[v,w]$

$P[w] = v$

~similarities to alignment



vector D. The algo. populates D.

step	v	C	D	cost from 1 to #
			2 3 4 5	
initialization	–	{2,3,4,5}	[50, 30, 100, 10]	
1	5	{2,3,4}	[50, 30, 20, 10]	
2	4	{2,3}	[40, 30, 20, 10]	
3	3	{2}	[35, 30, 20, 10]	

50 v. 50

20 v. 100

10 v. 10

∞ v. 30 ↖ improved for ④.

(D> → D=)

40 v. 50

∞ v. 30

$10 + 10 + 20 = 40 < 50$

① → ② → ③ → ④ D(w)

D(v)+L[v,w] ↳ D(w) := 40

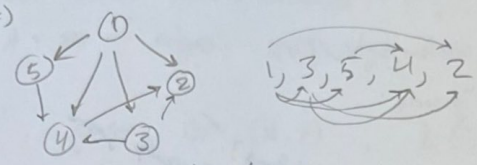
30+5 v. 50

Topological Sorting

a topological sort or ordering of a Directed graph is
a linear ordering of the vertices
s.t. for every directed edge
from vertex u to v, vertex u comes before v.

a topological ordering exists (iff) the graph is a DAG. (makes sense ~) deep structure.
↳ if exist, can be found in linear time.
Guides the fastest shortest path algo.

ex)



all arrows point right. ✓

Algorithm to find topological ordering: Topological Sorting algo.

for DAG $G = (V, E)$

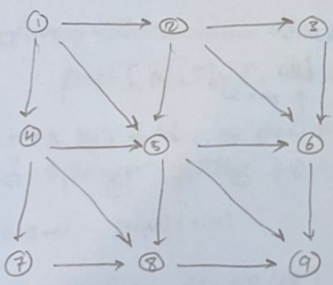
L = list of topologically sorted vertices of V (empty to begin?)

S = set of vertices w/ no incoming edges

While $S \neq \phi$, DO
Remove node u from S
Add u to L at the tail (end?)
For each node v w/ edge $e=(u,v) \in E$,
remove edge e from graph
if v has no incoming edges, | just update S
put v into S.

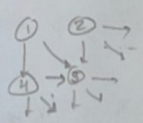
What if $S = \phi$ to start? is it impossible for a DAG?

ex)



$S = \{1, 2, 4, 3, 5, 6, 7, 8, 9\}$
$L = \{1, 2, 4, 3, 5, 6, 7, 8, 9\}$ → yields topological order.

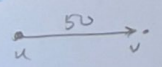
then remove $e=(u,v)$:



Now put ② → S. ✓ update S

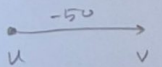
Needs review

Wt about longest path?



G shortest/longest path

=



$-G$ longest/shortest path. * only in DAGs though.

Next: most important algo hardest algo in course. will be tested.